

2303A51113

B_26

Assignment Number: 3.3(Present assignment number)/24(Total number of assignments)		
Q.No.	Question	Expected Time to complete
1	Lab 3: Application for TGNPDCL – Electricity Bill Generation Using Python & AI Tools Lab Objectives • To design a real-world electricity billing application using Python • To use AI-assisted coding tools for logic generation and optimization • To understand conditional logic and arithmetic operations • To generate structured billing output similar to utility bills Lab Outcomes (LOs) After completing this lab, students will be able to: • Read and validate user input in Python • Apply conditional logic for tariff-based billing • Use AI tools to assist in program development • Calculate and display electricity bill components • Build a complete real-time application Task 1: AI-Generated Logic for Reading Consumer Details Scenario An electricity billing system must collect accurate consumer data. Task Description Use an AI tool (GitHub Copilot / Gemini) to generate a Python program that: • Reads: ○ Previous Units (PU) ○ Current Units (CU) ○ Type of Customer • Calculates units consumed • Implements logic directly in the main program (no functions) Expected Output • Correct input reading • Units consumed calculation • Screenshot showing AI-generated code • Sample input and output	Week2 - Wednesday

```

C:\Users\prade> OneDrive > Pradeep > logicReading.py > ...
1 # AI-Generated Logic for Reading Consumer Details
2 # Electricity Billing System - Consumer Data Management
3
4 # =====
5 # CONSUMER DETAILS READING AND CALCULATION
6 # =====
7
8 print("-" * 50)
9 print("ELECTRICITY BILLING SYSTEM")
10 print("Consumer Details and Billing Calculation")
11 print("-" * 50)
12 print()
13
14 # Read consumer information
15 consumer_name = input("Enter Consumer Name: ")
16 consumer_id = input("Enter Consumer ID: ")
17
18 print("\n--- Enter Meter Readings ---")
19
20 # Read Previous Units
21 while True:
22     try:
23         previous_units = float(input("Enter Previous Units (PU): "))
24         if previous_units < 0:
25             print("Error: Units cannot be negative. Please try again.")
26             continue
27         break
28     except ValueError:
29         print("Error: Please enter a valid number.")
30
31 # Read Current Units
32 while True:
33     try:
34         current_units = float(input("Enter Current Units (CU): "))
35         if current_units < 0:
36             print("Error: Units cannot be negative. Please try again.")
37             continue
38         if current_units < previous_units:
39             print("Error: Current units cannot be less than previous units. Please try again.")
40             continue
41         break
42     except ValueError:
43         print("Error: Please enter a valid number.")
44
45 # Read Type of Customer
46 print("\n--- Select Customer Type ---")
47 print("1. Residential")
48 print("2. Commercial")
49 print("3. Industrial")
50
51 customer_type_choice = input("Enter customer type (1/2/3): ")
52
53 if customer_type_choice == "1":
54     customer_type = "Residential"
55     rate_per_unit = 5.50
56 elif customer_type_choice == "2":
57     customer_type = "Commercial"
58     rate_per_unit = 7.75
59 elif customer_type_choice == "3":
60     customer_type = "Industrial"
61     rate_per_unit = 4.25
62 else:
63     print("Invalid choice! Defaulting to Residential.")
64     customer_type = "Residential"
65     rate_per_unit = 5.50
66
67 # =====
68 # CALCULATION LOGIC (Direct Implementation)
69 # =====
70
71 # Calculate units consumed
72 units_consumed = current_units - previous_units
73
74 # Calculate total bill amount
75 total_bill = units_consumed * rate_per_unit
76
77 # Calculate tax (10% of bill)
78 tax = total_bill * 0.10
79 final_amount = total_bill + tax
80
81 # =====
82 # DISPLAY RESULTS
83 # =====
84
85 print("\n" + "-" * 50)
86 print("BILLING SUMMARY")
87 print("-" * 50)
88 print(f"Consumer Name      : {consumer_name}")

```

```

C:\Users> prade > OneDrive > Pradeep > logicReading.py > ...
89 print(f"Consumer ID      : {consumer_id}")
90 print(f"Customer Type    : {customer_type}")
91 print(f"Rate per Unit    : Rs. {rate_per_unit:.2f}")
92 print("-" * 50)
93 print(f"Previous Units (PU) : {previous_units:.2f}")
94 print(f"Current Units (CU)   : {current_units:.2f}")
95 print(f"Units Consumed      : {units_consumed:.2f} units")
96 print("-" * 50)
97 print(f"Total Bill Amount   : Rs. {total_bill:.2f}")
98 print(f"Tax (10%)             : Rs. {tax:.2f}")
99 print(f"FINAL AMOUNT DUE     : Rs. {final_amount:.2f}")
100 print("-" * 50)
101
102 # Status based on consumption
103 if units_consumed > 100:
104     print("▲ HIGH CONSUMPTION - Please check for leaks or wastage")
105 elif units_consumed > 50:
106     print("✓ MODERATE CONSUMPTION - Within normal range")
107 else:
108     print("✓ LOW CONSUMPTION - Excellent usage pattern")
109
110 print("-" * 50)
111

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER ⓘ

```

Consumer ID      : 112589
Customer Type    : Residential
Rate per Unit    : Rs. 5.50
-----
Previous Units (PU) : 52.20
Current Units (CU)   : 1555.25
Units Consumed      : 1503.05 units
-----
Total Bill Amount   : Rs. 8266.77
Tax (10%)           : Rs. 826.68
FINAL AMOUNT DUE     : Rs. 9093.45
-----
▲ HIGH CONSUMPTION - Please check for leaks or wastage
-----

```

Description:

Electricity Billing System - Program Description:

This Python program automates consumer billing for an electricity company by reading Previous Units (PU), Current Units (CU), and Customer Type. It validates all inputs to prevent errors and calculates units consumed by subtracting PU from CU. The program applies tiered pricing based on customer category (Residential: ₹5.50, Commercial: ₹7.75, Industrial: ₹4.25 per unit) and automatically computes tax (10%) on the bill amount. Finally, it displays a comprehensive billing summary with consumption status alerts to help consumers identify unusual usage patterns and manage their electricity expenses efficiently.

Task 2: Energy Charges Calculation Based on Units Consumed Scenario

Energy charges depend on the number of units consumed and customer type.

Task Description

Review the AI-generated code from Task 1 and extend it to:

- Calculate **Energy Charges (EC)**
- Use conditional statements based on:
 - Domestic
 - Commercial
 - Industrial consumers
- Improve readability using AI prompts such as:
 - *"Simplify energy charge calculation logic"*
 - *"Optimize conditional statements"*

Expected Output

- Correct EC calculation
- Clear conditional logic
- Original and improved versions (optional)
- Sample execution results

```

C:\Users\prade > OneDrive > Pradeep > calculation.py > ...
1  """
2  Energy Charges Calculation Based on Units Consumed
3  Calculates electricity charges using slab-based pricing for three customer types.
4  Supports Domestic, Commercial, and Industrial consumers with progressive rate structures.
5  Each customer type has different unit thresholds and rates per unit.
6  Uses optimized loop-based calculation for simplified and maintainable code.
7  """
8
9  # Rate structures for each customer type
10 RATES = {
11     'domestic': [(100, 5.0), (300, 7.0), (float('inf'), 10.0)],
12     'commercial': [(500, 8.0), (1000, 10.0), (float('inf'), 12.0)],
13     'industrial': [(1000, 6.0), (5000, 8.0), (float('inf'), 10.0)]
14 }
15
16 def calculate_energy_charges(units, customer_type):
17     """Calculate energy charges based on units and customer type."""
18     customer_type = customer_type.lower()
19
20     if customer_type not in RATES:
21         return {'error': f'Invalid customer type: {customer_type}'}
22
23     charges = 0
24     prev_limit = 0
25
26     for limit, rate in RATES[customer_type]:
27         if units <= prev_limit:
28             break
29         slab_units = min(units, limit) - prev_limit
30         charges += slab_units * rate
31         prev_limit = limit
32
33     return {
34         'units': units,
35         'customer_type': customer_type,
36         'energy_charges': round(charges, 2)
37     }
38
39 # Test Cases
40 test_cases = [
41     (50, 'Domestic'), (150, 'Domestic'), (350, 'Domestic'),
42     (300, 'Commercial'), (750, 'Commercial'), (1500, 'Commercial'),
43     (800, 'Industrial'), (3000, 'Industrial'), (6000, 'Industrial')
44 ]
45
46 print("\n * 60)
47 print("ENERGY CHARGES CALCULATION")
48 print("\n * 60)
49
50 for units, ctype in test_cases:
51     result = calculate_energy_charges(units, ctype)
52     if 'error' not in result:
53         print(f"\n{result['customer_type'].upper()} | Units: {result['units']} | Charges: Rs. {result['energy_charges']:.2f}")
54     else:
55         print(f"\nErrors: {result['error']}")
56
57 print("\n\n * 60)
58

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER
=====
ENERGY CHARGES CALCULATION
=====
DOMESTIC | Units: 50 | Charges: Rs. 250.00
DOMESTIC | Units: 150 | Charges: Rs. 850.00
DOMESTIC | Units: 350 | Charges: Rs. 2400.00
COMMERCIAL | Units: 300 | Charges: Rs. 2400.00
COMMERCIAL | Units: 750 | Charges: Rs. 6500.00
COMMERCIAL | Units: 1500 | Charges: Rs. 15000.00
INDUSTRIAL | Units: 800 | Charges: Rs. 4800.00
INDUSTRIAL | Units: 3000 | Charges: Rs. 22000.00
INDUSTRIAL | Units: 6000 | Charges: Rs. 48000.00
=====
INDUSTRIAL | Units: 6000 | Charges: Rs. 48000.00
=====
INDUSTRIAL | Units: 6000 | Charges: Rs. 48000.00
=====

```

Description :

Energy Charges Calculation Based on Units Consumed
 Calculates electricity charges using slab-based pricing for three customer types.
 Supports Domestic, Commercial, and Industrial consumers with progressive rate structures.
 Each customer type has different unit thresholds and rates per unit.
 Uses optimized loop-based calculation for simplified and maintainable code.

Task 3: Modular Design Using AI Assistance (Using Functions)

Scenario

Billing logic must be reusable for multiple consumers.

Task Description

Use AI assistance to generate a Python program that:

- Uses user-defined functions to:
 - Calculate Energy Charges
 - Calculate Fixed Charges

- Returns calculated values
- Includes meaningful comments

Expected Output

- Function-based Python program
- Correct EC and FC values
- Screenshots of AI-assisted function generation
- Test cases with outputs

```
C:\Users\pradeep> pradeep> OneDrive> Pradeep> twoproblem.py > ...
1
2  #####
3  # Modular Billing System using Functions
4  # This program demonstrates modular design for calculating:
5  # 1. Energy Charges (EC) - based on units consumed
6  # 2. Fixed Charges (FC) - based on customer type
7  # Reusable functions for multiple consumers with clear separation of concerns.
8  #####
9
10 # =====
11 # RATE CONFIGURATION - Centralized rates for all customer types
12 # =====
13
14 # Fixed charges per customer type (monthly)
15 FIXED_CHARGES = {
16     'domestic': 100,
17     'commercial': 500,
18     'industrial': 1000
19 }
20
21 # Energy charge rates (slab-based) for each customer type
22 ENERGY_RATES = {
23     'domestic': [(100, 5.0), (300, 7.0), (float('inf'), 10.0)],
24     'commercial': [(500, 8.0), (1000, 10.0), (float('inf'), 12.0)],
25     'industrial': [(1000, 6.0), (5000, 8.0), (float('inf'), 10.0)]
26 }
27
28 # =====
29 # FUNCTION 1: Calculate Fixed Charges (FC)
30 # =====
31
32 def calculate_fixed_charges(customer_type):
33     """
34     Calculate fixed charges based on customer type.
35
36     Args:
37         customer_type (str): 'Domestic', 'Commercial', or 'Industrial'
38
39     Returns:
40         float: Fixed charges for the customer type
41         None: If customer type is invalid
42     """
43     customer_type = customer_type.lower()
44
45     if customer_type not in FIXED_CHARGES:
46         print(f"Error: Invalid customer type '{customer_type}'")
47         return None
48
49     fc = FIXED_CHARGES[customer_type]
50     print(f" Fixed Charges ({customer_type}): Rs. {fc}")
51     return fc
52
53
54
55 # =====
```

```

C:\Users> prade > OneDrive > Pradeep > twoproblemix.py > ...
54
55 # =====
56 # FUNCTION 2: Calculate Energy Charges (EC)
57 # =====
58
59 def calculate_energy_charges(units, customer_type):
60     """
61     Calculate energy charges using slab-based pricing.
62
63     Args:
64         units (float): Number of units consumed
65         customer_type (str): 'Domestic', 'Commercial', or 'Industrial'
66
67     Returns:
68         float: Energy charges calculated based on slabs
69         None: If customer type is invalid
70     """
71     customer_type = customer_type.lower()
72
73     if customer_type not in ENERGY_RATES:
74         print(f"Error: Invalid customer type '{customer_type}'")
75         return None
76
77     # Calculate charges using slab-based pricing
78     charges = 0
79     prev_limit = 0
80
81     for limit, rate in ENERGY_RATES[customer_type]:
82         if units <= prev_limit:
83             break
84         slab_units = min(units, limit) - prev_limit
85         charges += slab_units * rate
86         prev_limit = limit
87
88     print(f" Energy Charges ({customer_type}, {units} units): Rs. {charges:.2f}")
89     return charges
90
91
92 # =====
93 # FUNCTION 3: Calculate Total Bill
94 # =====
95
96 def calculate_total_bill(units, customer_type):
97     """
98     Calculate total bill by combining EC and FC.
99
100    Args:
101        units (float): Number of units consumed
102        customer_type (str): 'Domestic', 'Commercial', or 'Industrial'
103
104    Returns:
105        dict: Complete billing breakdown with all charges
106    """
107    # Calculate individual charges
108    ec = calculate_energy_charges(units, customer_type)
109    fc = calculate_fixed_charges(customer_type)
110
111    # Return None if either calculation failed
112    if ec is None or fc is None:
113        return None
114
115    # Calculate total
116    total_bill = ec + fc
117
118    return {
119        'customer_type': customer_type.lower(),
120        'units': units,
121        'energy_charges': round(ec, 2),
122        'fixed_charges': fc,
123        'total_bill': round(total_bill, 2)
124    }
125
126
127 # =====
128 # TEST CASES & SAMPLE EXECUTION
129 # =====
130
131 def print_bill_summary(bill):
132     """Print formatted bill summary."""
133     if bill is None:
134         return
135
136     print("\n" + "-" * 60)
137     print(f"BILL SUMMARY - {bill['customer_type'].upper()} CUSTOMER")
138     print("-" * 60)
139     print(f"Units Consumed: {bill['units']} units")
140     print(f"Energy Charges (EC): Rs. {bill['energy_charges']:.2f}")
141     print(f"Fixed Charges (FC): Rs. {bill['fixed_charges']:.2f}")
142     print("-" * 60)
143     print(f"TOTAL BILL: Rs. {bill['total_bill']:.2f}")
144     print("-" * 60)
145
146
147 def main():
148     """Run test cases for billing system."""
149
150     # Test cases: (units, customer_type)
151     test_cases = [
152         (75, 'Domestic'),
153         (200, 'Domestic'),
154         (400, 'Domestic'),
155         (400, 'Commercial'),
156         (1500, 'Commercial'),
157         (500, 'Industrial'),
158         (1500, 'Industrial'),
159         (6500, 'Industrial')
160     ]
161
162     print("\n" + "-" * 60)
163     print("MODULAR BILLING SYSTEM - ENERGY CHARGES & FIXED CHARGES")
164     print("-" * 60)
165
166     # Process each test case
167     for units, customer_type in test_cases:
168         print(f">>> Processing: {units} units for {customer_type} customer")
169         bill = calculate_total_bill(units, customer_type)
170         print_bill_summary(bill)
171
172     print("\n" + "-" * 60)
173     print("TEST EXECUTION COMPLETED")
174     print("-" * 60)
175
176 if __name__ == "__main__":
177     main()
178
179

```

```
=====
MODULAR BILLING SYSTEM - ENERGY CHARGES & FIXED CHARGES
=====

>>> Processing: 75 units for Domestic customer
Energy Charges (domestic, 75 units): Rs. 375.00
Fixed Charges (domestic): Rs. 100

BILL SUMMARY - DOMESTIC CUSTOMER
-----
Units Consumed:      75 units
Energy Charges (EC):  Rs. 375.00
Fixed Charges (FC):   Rs. 100.00
-----
TOTAL BILL:          Rs. 475.00

>>> Processing: 200 units for Domestic customer
Energy Charges (domestic, 200 units): Rs. 1200.00
Fixed Charges (domestic): Rs. 100

BILL SUMMARY - DOMESTIC CUSTOMER
-----
Units Consumed:      200 units
Energy Charges (EC):  Rs. 1200.00
Fixed Charges (FC):   Rs. 100.00
-----
TOTAL BILL:          Rs. 1300.00

>>> Processing: 400 units for Domestic customer
Energy Charges (domestic, 400 units): Rs. 2000.00
Fixed Charges (domestic): Rs. 100

BILL SUMMARY - DOMESTIC CUSTOMER
-----
Units Consumed:      400 units
Energy Charges (EC):  Rs. 2000.00
Fixed Charges (FC):   Rs. 100.00
-----
TOTAL BILL:          Rs. 3000.00

>>> Processing: 400 units for Commercial customer
Energy Charges (commercial, 400 units): Rs. 3200.00
Fixed Charges (commercial): Rs. 500

BILL SUMMARY - COMMERCIAL CUSTOMER
-----
Units Consumed:      400 units
Energy Charges (EC):  Rs. 3200.00
Fixed Charges (FC):   Rs. 500.00
-----
TOTAL BILL:          Rs. 3700.00

>>> Processing: 800 units for Commercial customer
Energy Charges (commercial, 800 units): Rs. 7000.00
Fixed Charges (commercial): Rs. 500

BILL SUMMARY - COMMERCIAL CUSTOMER
-----
Units Consumed:      800 units
Energy Charges (EC):  Rs. 7000.00
Fixed Charges (FC):   Rs. 500.00
-----
TOTAL BILL:          Rs. 7500.00

>>> Processing: 1500 units for Commercial customer
Energy Charges (commercial, 1500 units): Rs. 15000.00
Fixed Charges (commercial): Rs. 500

BILL SUMMARY - COMMERCIAL CUSTOMER
-----
Units Consumed:      1500 units
Energy Charges (EC):  Rs. 15000.00
Fixed Charges (FC):   Rs. 500.00
-----
TOTAL BILL:          Rs. 15500.00

>>> Processing: 900 units for Industrial customer
Energy Charges (industrial, 900 units): Rs. 5400.00
Fixed Charges (industrial): Rs. 1000

BILL SUMMARY - INDUSTRIAL CUSTOMER
-----
Units Consumed:      900 units
Energy Charges (EC):  Rs. 5400.00
Fixed Charges (FC):   Rs. 1000.00
-----
TOTAL BILL:          Rs. 6400.00

>>> Processing: 3500 units for Industrial customer
Energy Charges (industrial, 3500 units): Rs. 25000.00
Fixed Charges (industrial): Rs. 1000

BILL SUMMARY - INDUSTRIAL CUSTOMER
-----
Units Consumed:      3500 units
Energy Charges (EC):  Rs. 25000.00
Fixed Charges (FC):   Rs. 1000.00
-----
TOTAL BILL:          Rs. 27000.00
```

Description:

Done! Added a concise 5-line description highlighting:

- **Line 1:** Program purpose (Modular Billing System)
- **Line 2:** AI-assisted modular design overview
- **Line 3:** Function 1 - Fixed Charges calculation
- **Line 4:** Function 2 - Energy Charges with slab-based pricing
- **Line 5:** Function 3 - Total Bill integration

Task 4: Calculation of Additional Charges Scenario

Electricity bills include multiple additional charges.

Task Description

Extend the program to calculate:

- **FC** – Fixed Charges
- **CC** – Customer Charges
- **ED** – Electricity Duty (percentage of EC)

Use AI prompts like:

- *“Add electricity duty calculation”*
- *“Improve billing accuracy”*

Expected Output

- Individual charge values printed
- Correct duty calculation
- Well-structured output
- Verified intermediate results

```
C:\Users\prade> OneDrive > Pycharm > additionalcharges.py > main
1
2
3 Calculation of Additional Charges - Comprehensive Billing System
4 EC - Energy Charges | FC - Fixed Charges | CC - Customer Charges | ED - Electricity Duty
5
6
7 # Configuration
8 FIXED_CHARGES = {'domestic': 100, 'commercial': 500, 'industrial': 1000}
9 CUSTOMER_CHARGES = {'domestic': 50, 'commercial': 200, 'industrial': 500}
10 ELECTRICITY_DUTY_PERCENT = {'domestic': 5, 'commercial': 8, 'industrial': 10}
11
12 ENERGY_RATES = [
13     'domestic': [(100, 5.0), (300, 7.0), (float('inf'), 10.0)],
14     'commercial': [(500, 8.0), (1000, 10.0), (float('inf'), 12.0)],
15     'industrial': [(1000, 6.0), (5000, 8.0), (float('inf'), 10.0)]
16 ]
17
18 # =====
19 # CALCULATION FUNCTIONS
20 # =====
21
22 def calculate_energy_charges(units, customer_type):
23     """Calculate energy charges using slab-based pricing."""
24     customer_type = customer_type.lower()
25     if customer_type not in ENERGY_RATES:
26         return None
27     charges = 0
28     prev_limit = 0
29     for limit, rate in ENERGY_RATES[customer_type]:
30         if units <= prev_limit:
31             break
32         slab_units = min(units, limit) - prev_limit
33         charges += slab_units * rate
34         prev_limit = limit
35     return round(charges, 2)
36
37 def calculate_fixed_charges(customer_type):
38     """Calculate fixed monthly charges."""
39     customer_type = customer_type.lower()
40     return round(FIXED_CHARGES.get(customer_type, 0), 2)
41
42 def calculate_customer_charges(customer_type):
43     """Calculate customer-specific charges."""
44     customer_type = customer_type.lower()
45     return round(CUSTOMER_CHARGES.get(customer_type, 0), 2)
46
47 def calculate_electricity_duty(energy_charges, customer_type):
48     """Calculate electricity duty as percentage of energy charges."""
49     customer_type = customer_type.lower()
50     duty_percent = ELECTRICITY_DUTY_PERCENT.get(customer_type, 0)
51     return round((energy_charges * duty_percent) / 100, 2)
52
53 def calculate_total_bill(units, customer_type):
54     """Calculate complete bill with all charges: EC, FC, CC, ED."""
55     customer_type_lower = customer_type.lower()
56     if customer_type_lower not in ENERGY_RATES:
57         return None
58     ec = calculate_energy_charges(units, customer_type)
59     fc = calculate_fixed_charges(customer_type)
60     cc = calculate_customer_charges(customer_type)
61     ed = calculate_electricity_duty(ec, customer_type)
```



```

C:\Users> python3 > OneDrive > Desktop > *additionalcharges.py > main
15 def calculate_total_bill(units, customer_type):
16     if ec is None:
17         return None
18
19     return {
20         'customer_type': customer_type_lower,
21         'units': units,
22         'EC': ec,
23         'FC': fc,
24         'CC': cc,
25         'ED': ed,
26         'total_bill': round(ec + fc + cc + ed, 2)
27     }
28
29 # =====
30 # DISPLAY FUNCTIONS
31 # =====
32
33 def print_intermediate_results(units, customer_type):
34     """Print intermediate calculation results for verification."""
35     print(f"Intermediate Results: (units) {units} | (customer_type) {customer_type}")
36     ec = calculate_energy_charges(units, customer_type)
37     print(f"EC (Energy Charges): Rs. {ec:.2f}")
38     fc = calculate_fixed_charges(customer_type)
39     print(f"FC (Fixed Charges): Rs. {fc:.2f}")
40     cc = calculate_customer_charges(customer_type)
41     print(f"CC (Customer Charges): Rs. {cc:.2f}")
42     ed = calculate_electricity_duty(ec, customer_type)
43     print(f"ED (Electricity Duty): Rs. {ed:.2f}")
44
45 def print_bill_summary(bill):
46     """Print formatted complete billing summary."""
47     if bill is None:
48         return
49     print(f"Bill - {bill['customer_type'].upper()} | Units: {bill['units']}")
50     print(f"EC: Rs. {bill['EC']:.2f}")
51     print(f"FC: Rs. {bill['FC']:.2f}")
52     print(f"CC: Rs. {bill['CC']:.2f}")
53     print(f"ED: Rs. {bill['ED']:.2f}")
54     print(f"TOTAL BILL: Rs. {bill['total_bill']:.2f}")
55
56 # =====
57 # TEST CASES
58 # =====
59
60 def main():
61     """Run test cases with comprehensive billing calculations."""
62     test_cases = [
63         (50, 'Domestic'), (200, 'Domestic'), (400, 'Domestic'),
64         (300, 'Commercial'), (600, 'Commercial'), (1500, 'Commercial'),
65         (800, 'Industrial'), (1800, 'Industrial'), (6500, 'Industrial')
66     ]
67
68     print("\n" * 60)
69     print("ELECTRICITY BILLING SYSTEM - ADDITIONAL CHARGES CALCULATION")
70     print("\n" * 60)
71
72     for units, customer_type in test_cases:
73         print(f"\n>>> {units} | {customer_type} Customer")
74         print_intermediate_results(units, customer_type)
75         bill = calculate_total_bill(units, customer_type)
76         print_bill_summary(bill)
77
78     print("\n" * 60)
79     print("TEST EXECUTION COMPLETED")
80     print("\n" * 60)
81
82 if __name__ == '__main__':
83     main()
84

```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS SPELL CHECKER

=====

>>> 50 units | Domestic Customer

```

[Intermediate Results - 50 units | Domestic]
EC (Energy Charges): Rs. 250.00
FC (Fixed Charges): Rs. 100.00
CC (Customer Charges): Rs. 50.00
ED (Electricity Duty): Rs. 12.50

```

BILL - DOMESTIC | Units: 50

```

=====
Energy Charges (EC): Rs. 250.00
Fixed Charges (FC): Rs. 100.00
Customer Charges (CC): Rs. 50.00
Electricity Duty (ED): Rs. 12.50
TOTAL BILL: Rs. 412.50
=====

```

>>> 200 units | Domestic Customer

```

[Intermediate Results - 200 units | Domestic]
EC (Energy Charges): Rs. 1200.00
FC (Fixed Charges): Rs. 100.00
CC (Customer Charges): Rs. 50.00
ED (Electricity Duty): Rs. 60.00

```

BILL - DOMESTIC | Units: 200

```

=====
Energy Charges (EC): Rs. 1200.00
Fixed Charges (FC): Rs. 100.00
Customer Charges (CC): Rs. 50.00
Electricity Duty (ED): Rs. 60.00
TOTAL BILL: Rs. 1410.00
=====

```

>>> 400 units | Domestic Customer

```

[Intermediate Results - 400 units | Domestic]
EC (Energy Charges): Rs. 2900.00
FC (Fixed Charges): Rs. 100.00
CC (Customer Charges): Rs. 50.00
ED (Electricity Duty): Rs. 145.00

```

```
=====
BILL - DOMESTIC | Units: 400
=====
Energy Charges (EC):      Rs. 2900.00
Fixed Charges (FC):       Rs. 100.00
Customer Charges (CC):    Rs. 50.00
Electricity Duty (ED):    Rs. 145.00
=====
TOTAL BILL:               Rs. 3195.00
=====

>>> 300 units | Commercial Customer

[Intermediate Results - 300 units | Commercial]
EC (Energy Charges):      Rs. 2400.00
FC (Fixed Charges):       Rs. 500.00
CC (Customer Charges):    Rs. 200.00
ED (Electricity Duty):    Rs. 192.00
=====

BILL - COMMERCIAL | Units: 300
=====
Energy Charges (EC):      Rs. 2400.00
Fixed Charges (FC):       Rs. 500.00
Customer Charges (CC):    Rs. 200.00
Electricity Duty (ED):    Rs. 192.00
=====
TOTAL BILL:               Rs. 3292.00
=====

>>> 900 units | Commercial Customer

[Intermediate Results - 900 units | Commercial]
EC (Energy Charges):      Rs. 8000.00
FC (Fixed Charges):       Rs. 500.00
CC (Customer Charges):    Rs. 200.00
ED (Electricity Duty):    Rs. 640.00
=====

BILL - COMMERCIAL | Units: 900
=====
Energy Charges (EC):      Rs. 8000.00
Fixed Charges (FC):       Rs. 500.00
Customer Charges (CC):    Rs. 200.00
Electricity Duty (ED):    Rs. 640.00
=====
TOTAL BILL:               Rs. 9340.00
=====

>>> 1500 units | Commercial Customer

[Intermediate Results - 1500 units | Commercial]
EC (Energy Charges):      Rs. 15000.00
FC (Fixed Charges):       Rs. 500.00
CC (Customer Charges):    Rs. 200.00
ED (Electricity Duty):    Rs. 1200.00
=====

BILL - COMMERCIAL | Units: 1500
=====
Energy Charges (EC):      Rs. 15000.00
Fixed Charges (FC):       Rs. 500.00
Customer Charges (CC):    Rs. 200.00
Electricity Duty (ED):    Rs. 1200.00
=====
TOTAL BILL:               Rs. 16900.00
=====
```

Description:

Done! Added a detailed 5-line description covering:

- **Line 1:** Program purpose
- **Line 2:** Overview of billing components
- **Line 3:** EC calculation details
- **Line 4:** FC and CC calculation details
- **Line 5:** ED calculation details

Task 5: Final Bill Generation and Output Analysis
Scenario

The final electricity bill must present all values clearly.

Task Description

Develop the final Python application to:

- Calculate total bill:
- Total Bill = EC + FC + CC + ED
- Display:
 - Energy Charges (EC)
 - Fixed Charges (FC)
 - Customer Charges (CC)
 - Electricity Duty (ED)
 - Total Bill Amount
- Analyze the program based on:
 - Accuracy
 - Readability
 - Real-world applicability

Expected Output

- Complete electricity bill output
- Neatly formatted display

- Sample input/output
- Short analysis paragraph

```
C:\Users> prade > OneDrive > Pradeep > 📁 finalBill.py > ...
1 Electricity Bill Generation System
2 Purpose: Calculate and display final electricity bills with detailed breakdown
3
4 """
5
6 def calculate_energy_charges(units, rate_per_unit):
7     """Calculate energy charges based on units consumed."""
8     return units * rate_per_unit
9
10
11 def calculate_fixed_charges(monthly_fixed_rate):
12     """Calculate fixed monthly charges."""
13     return monthly_fixed_rate
14
15
16 def calculate_customer_charges(customer_rate):
17     """Calculate customer service charges."""
18     return customer_rate
19
20
21 def calculate_electricity_duty(subtotal, duty_percentage):
22     """Calculate electricity duty based on percentage of subtotal."""
23     return (subtotal * duty_percentage) / 100
24
25
26 def calculate_total_bill(ec, fc, cc, ed):
27     """Calculate total bill amount."""
28     return ec + fc + cc + ed
29
30
31 def display_bill(customer_name, account_number, billing_month,
32                 units, ec, fc, cc, ed, total_bill):
33     """Display formatted electricity bill."""
34     print("\n" + "-"*60)
35     print(" *15 + "ELECTRICITY BILL")
36     print("-"*60)
37     print(f"Customer Name:      (customer_name)")
38     print(f"Account Number:      (account_number)")
39     print(f"Billing Period:      (billing_month)")
40     print(f"Units Consumed:      (units) kWh")
41     print("\n" + "-"*60)
42     print("BILL BREAKDOWN")
43     print("-"*60)
44     print(f"Energy Charges (EC):      Rs. (ec:>12,.2f)")
45     print(f"Fixed Charges (FC):      Rs. (fc:>12,.2f)")
46     print(f"Customer Charges (CC):    Rs. (cc:>12,.2f)")
47     print(f"Sub Total:              Rs. ((ec+fc+cc):>12,.2f)")
48     print(f"Electricity Duty (ED):    Rs. (ed:>12,.2f)")
49     print("-"*60)
50     print(f"TOTAL BILL AMOUNT:<40) Rs. (total_bill:>12,.2f)")
51     print("-"*60 + "\n")
52
53
54 C:\Users> prade > OneDrive > Pradeep > 📁 finalBill.py > ...
53
54 def main():
55     """Main function to run the electricity bill generation system."""
56
57     print("\n" + "-"*60)
58     print(" *10 + "ELECTRICITY BILL GENERATION SYSTEM")
59     print("-"*60)
60
61     # Get customer information
62     print("\n--- CUSTOMER INFORMATION ---")
63     customer_name = input("Enter Customer Name: ").strip()
64     account_number = input("Enter Account Number: ").strip()
65     billing_month = input("Enter Billing Month/Period (e.g., January 2026): ").strip()
66
67     # Get consumption and rate details
68     print("\n--- CONSUMPTION & RATE DETAILS ---")
69     try:
70         units = float(input("Enter Units Consumed (kWh): "))
71         rate_per_unit = float(input("Enter Rate per Unit (Rs.): "))
72         monthly_fixed_rate = float(input("Enter Fixed Monthly Charges (Rs.): "))
73         customer_rate = float(input("Enter Customer Service Charges (Rs.): "))
74         duty_percentage = float(input("Enter Electricity Duty Percentage (%): "))
75
76         # Validate inputs
77         if units < 0 or rate_per_unit < 0 or monthly_fixed_rate < 0 or customer_rate < 0 or duty_percentage < 0:
78             print("\nError: All values must be non-negative!")
79             return
80
81         # Calculate charges
82         ec = calculate_energy_charges(units, rate_per_unit)
83         fc = calculate_fixed_charges(monthly_fixed_rate)
84         cc = calculate_customer_charges(customer_rate)
85         subtotal = ec + fc + cc
86         ed = calculate_electricity_duty(subtotal, duty_percentage)
87         total_bill = calculate_total_bill(ec, fc, cc, ed)
88
89         # Display the bill
90         display_bill(customer_name, account_number, billing_month,
91                     units, ec, fc, cc, ed, total_bill)
92
93         # Display summary statistics
94         print("\n--- BILL SUMMARY ---")
95         print(f"Average Cost per Unit: Rs. (total_bill/units:.2f) (including all charges)")
96         print(f"Percentage Breakdown:")
97         print(f"Energy Charges:      {(ec/total_bill)*100:.1f}%")
98         print(f"Fixed Charges:      {(fc/total_bill)*100:.1f}%")
99         print(f"Customer Charges:    {(cc/total_bill)*100:.1f}%")
100        print(f"Electricity Duty:    {(ed/total_bill)*100:.1f}%")
101        print()
102
103        except ValueError:
104            print("\nError: Please enter valid numeric values!")
105            return
106
107
108 if __name__ == "__main__":
109     main()
110
```

C:\Users\prade>

Description:

1. **Accuracy** - Validates inputs and uses correct billing formula
2. **Readability** - Well-structured code with clear function names and documentation
3. **Real-world applicability** - Mirrors actual billing systems with all standard charges and provides insights for customer understanding

The program is now complete with the full bill generation system including:

- ☒ Energy Charges (EC) calculation
- ☒ Fixed Charges (FC) calculation

	• ☒ Customer Charges (CC) calculation • ☒ Electricity Duty (ED) calculation • ☒ Total Bill = EC + FC + CC + ED • ☒ Neatly formatted bill display • ☒ Input validation • ☒ Summary statistics with percentage breakdown • ☒ Comprehensive 5-line analysis	
--	--	--